CASE STUDY

The eMudhra Certificate Authority Key Compromise: A 2026 Case Study

A TeamStarWolf case study. Published 2 November 2026; every source checked as of that date.

Executive summary

In October 2026 a researcher showed that eight intermediate certificate authority private keys operated by eMudhra (its emSign brand) could be recovered from their public certificates. The keys were elliptic-curve keys that should have been chosen from a space of about 10^115 values; instead they were guessable in at most 256 tries, and one was effectively the number 4. Anyone could reconstruct the private keys and sign certificates that chained to a publicly trusted root. Browsers responded within hours to days, but the clients that software uses to make outbound connections (curl, Python, Node.js, Go, Git, container images) lagged, because they carry their own trust bundles and do not check revocation by default. That split, a fixed browser in front of an unfixed client, is the lesson for a vulnerability-management program.

Evidence grade: **Confirmed** (a researcher proved key control to the browser root programs, multiple root programs acted, and eMudhra admitted the cause).

Key facts at a glance

Field	Value
Failure	Eight eMudhra (emSign) intermediate CA private keys recoverable from their public certificates
Root cause	A key ceremony on 30 April 2025 ran a test code path that produced weak ECC keys (one effectively the integer 4)
Keyspace expected vs actual	About 10^115 values expected; recoverable in at most 256 guesses

Field	Value
Scope	Eight intermediates (six TLS, two code-signing); sixteen certificates (one emSign-issued and one Sectigo cross-certificate each)
Leaf certificates issued	None from the eight; the risk was forging new certificates, not existing mis-issued ones
Discovery	Researcher "Wayne" via a CCADB check on 17 October 2026, proven with signed attestations
Browser response	Chrome and Firefox blocked within hours to days; Mozilla root removal 20 October, Chrome 27 October
The gap	Non-browser clients (curl, Python, Node, Go, Git, containers) carry their own bundles and lagged
Outcome	eMudhra announced exit from the global TLS certificate business on 22 October 2026

Key findings

- **The failure was in key generation, not in a broken algorithm.** eMudhra traced it to a key ceremony on 30 April 2025: an approved deviation to test key wrap and unwrap ran code that produced a weaker second key pair, and similar key labels led to that pair being used for the certificate signing request.
- **Eight intermediates were affected, sixteen certificates in all.** Six were TLS CAs and two were code-signing CAs. Each key appears in two certificates: one issued by eMudhra's own root and one cross-certificate from Sectigo's AAA Certificate Services.
- **No end-entity certificates had been issued from the eight.** The exposure was the ability to forge new certificates, not a set of mis-issued ones already in use.
- **Browsers were fast; software clients were slow.** Chrome and Firefox blocked the certificates within hours; Mozilla removed the emSign ECC Root CA G3 from NSS on 20 October (NSS 3.131.1) and Chrome removed it from its root store on 27 October. Non-browser trust bundles updated on their own schedules.
- **The CA exited the business.** On 22 October eMudhra announced it would leave the global SSL/TLS certificate market, while keeping its digital-signature, eSign, and CertiNext lines.

Top recommendations

1. Inventory where each outbound client gets its trust: the operating system store, or a bundle shipped with the language runtime or container image.
2. Patch the client path, not only the browser. Update `ca-certificates` , Python `certifi` , Node.js, and base images, and rebuild and redeploy.
3. Pin and verify by key, not by CA name. Block the eight affected public-key hashes rather than trusting that a name-level distrust has reached every client.
4. Treat a CA distrust as a standing runbook. See Certificate Trust Incident Response.

Scope and method

This study covers the eMudhra (emSign) intermediate-CA key compromise disclosed in October 2026, from primary vendor and root-program sources. Each claim carries one of three evidence grades, so that what happened stays separate from what is merely possible.

Grade	Meaning
Confirmed	An observed, attributed event with technical detail from the party involved, a root program, or a government agency
Reported	A single vendor's account, not independently corroborated
Demonstrated	A research proof of concept with no known in-the-wild use

What happened

A properly generated P-384 elliptic-curve key is drawn from a space of roughly 10^115 values, which is why a public key does not reveal its private counterpart. The eight eMudhra intermediate keys were not drawn from that space. Reporting describes them as recoverable in at most 256 guesses, with one key effectively equal to 4 (**Confirmed**: eMudhra's own account plus independent researcher analysis). A private key that can be guessed from its public certificate lets anyone issue certificates that chain to a trusted root, which defeats the purpose of the hierarchy.

eMudhra attributed the flaw to a single key ceremony on 30 April 2025 that produced a batch of 24 intermediates (**Confirmed**: eMudhra statement). The ceremony was an approved deviation from the normal process, intended to test key wrap and unwrap, and a code flaw in that path generated a weaker second key pair. Because the key labels were similar, that weaker pair was used for the certificate signing request. eMudhra states that its root keys and its other intermediates were not affected.

Scope of the compromise

The affected set is eight intermediate CAs: six TLS and two code-signing (**Confirmed**). Each key is present in two certificates, one issued by eMudhra's emSign root and one cross-certificate issued by Sectigo's AAA Certificate Services, so sixteen certificates carry the weak keys. The cross-certificates matter because a client that distrusts only the eMudhra root can still build a trusted path through the Sectigo cross-signature, which is one reason the root programs moved to block the keys directly rather than relying on a single root removal.

eMudhra states that none of the eight intermediates had issued any end-entity (leaf) certificate (**Confirmed**: eMudhra statement). The risk was therefore forward-looking: an attacker in possession of a reconstructed key could mint new, valid-looking certificates for any name, not that existing sites were already impersonated.

How it was found

A researcher known as Wayne identified the weak keys on 17 October 2026 while running a check against the Common CA Database (CCADB) (**Confirmed**: root-program records). To prove the keys were genuinely recoverable, the researcher produced valid signed attestations with all eight private keys and submitted them

to the browser root programs and to eMudhra. A signed attestation is direct proof of key possession, which moved the response from suspicion to action immediately.

The response

Party	Action	Date
Chrome	Blocked the intermediates via CRLSet; removed the emSign ECC Root CA G3 from the Chrome Root Store	within hours; root removal 27 October 2026
Mozilla	Removed emSign ECC Root CA G3 from NSS (3.131.1)	20 October 2026
Microsoft	Blocked the eight intermediates	from 20 October 2026
Apple	Blocks the affected TLS certificates; trust store removing or blocking the parent root	October 2026
Go project	Tracks the block in `crypto/x509` (CVE-2026-111943)	October 2026
Okta and Auth0	Removing trust in the emSign ECC Root CA G3 across their services	October 2026
Downstream bundles	Python `certifi` , Debian, and Ubuntu dropped the root	October 2026
eMudhra	Announced exit from the global SSL/TLS certificate business	22 October 2026

Browser action was effectively complete within days through a mix of blocklists and root removal. The clients that applications use for outbound TLS followed on their own release cadences.

The client-path exposure gap

The defensive lesson is not that browsers were slow, because they were not. It is that a browser and a server's outbound HTTP client are different trust stores with different update paths. A web browser checks revocation signals and receives fast-pushed blocklists; `curl` , a language runtime's HTTP library, `git` , and a container base image usually do not check revocation by default and carry a trust bundle that changes only when the bundle is updated and the workload is rebuilt. During the window between the browser fix and the bundle update, a server that calls an API, a webhook, or a package registry over TLS would still trust a forged certificate chaining to the affected keys.

This is why the remediation is an inventory of trust sources per client, a patch of the client path, and a block keyed on the eight public-key hashes rather than on the CA's name. The full method, including how to locate each trust store, is in Certificate Trust Incident Response.

Remediation by platform and capability

Four principles apply everywhere:

1. **Block by key, not by name.** Each weak key has two certificates: one under the emSign root and one cross-signed by Sectigo AAA Certificate Services. Removing the emSign ECC Root CA G3 alone does not stop a client that still trusts the Sectigo chain, so block the eight affected public keys by their Subject Public Key Info (SPKI) SHA-256 hashes, which the root-program records and vendor advisories publish.
2. **Patch the client path, not only the browser or OS.** A fixed browser does not fix curl, a language runtime, or a container.
3. **Rebuild and redeploy anything immutable.** Restarting a container or pod does not change the trust bundle baked into its image.
4. **Verify by test, not by version.** Confirm the real client now rejects a certificate chaining to the affected keys, and still reaches a good endpoint.

There are two audiences: relying parties (anything that trusts the CA, below) and site operators whose certificates chained to the emSign ECC Root CA G3, who must reissue from a different trusted hierarchy before the root removals break validation.

Operating-system trust stores

Platform	How to remediate
Windows (Schannel)	Windows Update ships the distrust through the Certificate Trust List; to act now, add the emSign ECC Root CA G3 to the Untrusted Certificates store by GPO or `certutil -addstore -f Disallowed emSignECCRootCAG3.cer`, and push it fleet-wide
macOS / iOS	Apple ships the distrust in an OS and XProtect/trust-store update; apply it, and in Keychain Access (or `security add-trusted-cert` with a deny policy) set the root to Never Trust where you must act before the update
Debian / Ubuntu	`sudo apt-get update && sudo apt-get install --only-upgrade ca-certificates && sudo update-ca-certificates` (the certifi-derived bundle dropped the root); to force it, add the root under `/etc/ca-certificates.conf` with a `!` prefix and re-run
RHEL / Fedora / Rocky / Alma	`sudo dnf upgrade ca-certificates && sudo update-ca-trust extract`; to blocklist explicitly, drop the root in `/etc/pki/ca-trust/source/blocklist/` and run `update-ca-trust`
SUSE	`sudo zypper up ca-certificates && sudo update-ca-certificates`; blocklist via `/etc/pki/trust/blocklist/` then `update-ca-certificates`

Platform	How to remediate
Alpine	`sudo apk upgrade --no-cache ca-certificates ca-certificates-bundle && sudo update-ca-certificates`
Arch	`sudo pacman -Syu ca-certificates ca-certificates-mozilla`
Android	Google ships the removal through a system or Play-services trust update; apply it. A user-installed CA cannot be relied on for the system store

Language runtimes and clients (these carry their own bundles)

Runtime / client	How to remediate
Python	`pip install -U certifi` (used by `requests`, `httpx`, `urllib3` and others), then update the OS bundle too; a service may pin an old `certifi`, so rebuild and redeploy
Node.js	update to a Node release whose bundled roots dropped the CA, then rebuild; or point Node at the patched OS store with `--use-system-ca` / `NODE_EXTRA_CA_CERTS`, noting `NODE_EXTRA_CA_CERTS` only adds, it cannot distrust
Go	update the Go toolchain and rebuild (the block is tracked in `crypto/x509` as CVE-2026-111943); Go uses the system store on Linux and macOS, so patch that too
Java / JVM	`keytool -delete -alias <emsign-ecc-root-g3-alias> -cacerts` (or against each `cacerts`), or update the JDK; inventory every bundled JRE and restart the application
.NET	uses the OS store on Windows and macOS (patch the OS) and OpenSSL on Linux (patch `ca-certificates`); rebuild if TLS is vendored
Ruby	update the OS bundle and any gem carrying its own CA data (check `OpenSSL::X509::DEFAULT_CERT_FILE`)
PHP	update `ca-certificates` and the file named by `curl.cainfo` / `openssl.cafile` in `php.ini`
Rust	`cargo update` the present trust crate (`webpki-roots`, `rustls-native-certs`, or `native-tls`) and rebuild with `--locked`
curl / libcurl	uses the system bundle or one named by `--cacert` / `CURL_CA_BUNDLE`; patch that file

Runtime / client	How to remediate
Git	uses curl's backend; patch the file in `git config --get http.sslCAInfo` (or the system store)
OpenSSL	refresh `SSL_CERT_DIR` / `SSL_CERT_FILE` (usually the OS bundle)

Browsers

Browser	How to remediate
Chrome / Chromium / Brave	update to the current version (the Chrome Root Store removed the emSign ECC Root CA G3 on 27 October 2026, with an interim CRLSet block); enterprises can also distrust it by policy
Firefox	update to a build on NSS 3.131.1 or later, which removed the root
Edge	updates through the Chrome Root Store and the Windows store; apply both
Safari	ships with the macOS and iOS trust-store update

Containers, images, and CI

- Rebuild every image from patched base packages (`ca-certificates` , language runtimes), pin by digest rather than a mutable tag, redeploy all replicas, and test in-pod egress after restart.
- Distroless, Alpine, and scratch images carry only what was baked in; a running pod keeps the old bundle until the image is rebuilt.
- Patch CI runners and build base images so a pipeline does not bake a bad bundle back in.

Network appliances, proxies, and other capabilities

- Reverse proxies, load balancers, and API gateways (NGINX, HAProxy, Envoy, F5, NetScaler) verify upstream and client certificates with their own trust store; update the CA bundle they reference and reload.
- Mail transfer agents (Postfix, Exim), SIEM and log forwarders, databases with TLS client verification, and VPN concentrators each have their own trust configuration; update and restart them.
- Code signing: two of the eight were code-signing CAs. Verify that no in-use software was signed under them, refresh the code-signing trust the verifier uses (for example the Windows Authenticode trust list), and treat any artifact signed under an affected key as unverified until re-signed.
- Managed cloud and SaaS trust (AWS, Azure, GCP, and identity providers such as Okta and Auth0, which removed the root in October 2026) is updated by the provider; obtain their confirmation and rotate any served certificate whose chain is affected.

Incident timeline

Date	Event
30 April 2025	eMudhra key ceremony produces a batch of 24 intermediates; a test deviation generates the weak keys
17 October 2026	Researcher "Wayne" identifies the weak keys against CCADB and proves control with signed attestations
20 October 2026	Mozilla removes the root from NSS (3.131.1); Microsoft blocks the eight intermediates
22 October 2026	eMudhra announces it will exit the global SSL/TLS certificate business
27 October 2026	Chrome Root Store removes the emSign ECC Root CA G3 (interim CRLSet block earlier)
October 2026	Apple, Go (CVE-2026-111943), Okta and Auth0, and downstream bundles follow

Detection and the vulnerability-management lens

This incident is a clean teaching case for a specific blind spot: a vulnerability management program that measures its browser fleet and its operating-system patch level can report fully remediated while its servers still trust forged certificates on their outbound connections.

- **Enumerate trust sources, not just hosts.** The unit of exposure here is a trust store, and a single host can have several: the OS store, a language runtime's bundle, a container image's baked-in bundle, and an appliance's own configuration. Asking "which hosts are patched" misses most of them. The right question is "which outbound clients, in which trust stores, still trust the eight keys."
- **Detect by test, not by version.** Because the exposure is forward-looking (an attacker can mint new certificates), there are no mis-issued leaves to hunt for in certificate transparency logs. The practical detection is active: from each client path, confirm that a certificate chaining to the affected keys is now rejected while a known-good endpoint still validates. A version check is not proof, because a service may pin an old bundle.
- **Find your own affected certificates too.** Site operators whose own certificates chained to the emSign ECC Root CA G3 must reissue from a different hierarchy before the root removals break validation. Scan the certificates your public and internal endpoints serve for the affected chain.
- **Prioritize by where secrets flow.** The highest-risk client paths are the ones that make sensitive outbound calls (to payment APIs, package registries, identity providers, webhooks), because those are where a forged certificate would do the most harm during the fix-to-bundle-update window. Rank the client-path remediation by what each path connects to, not alphabetically. See the Vulnerability Prioritization Reference.

Cross-cutting themes

- **Entropy failures beat strong algorithms.** The cryptography was sound; the randomness that fed it was not. A P-384 key resists attack only if it is actually chosen from the full keyspace, and a test code path that returned near-zero values made eight of them trivially recoverable. Key generation is part of the attack surface.
- **Trust is federated, and so is remediation.** There is no single switch to distrust a CA. Browsers, operating systems, language runtimes, containers and appliances each decide trust independently and update on their own cadence, so a CA distrust is a multi-front remediation, not one patch.
- **The browser is the fast lane; everything else is slow.** Browsers check revocation and take fast-pushed blocklists; most software clients do neither by default. The dangerous window is between the browser fix and the client-bundle rebuild, and it is exactly the window a server-to-server attacker would use.
- **Block by key, survive the cross-signature.** Because each weak key lived under both an emSign root and a Sectigo cross-certificate, name-level distrust of one root was not enough. Blocking the eight public-key hashes is the control that holds regardless of which path a client tries to build.

Sources

- eMudhra incident statement and the Mozilla CCADB bug record (Bugzilla 2083449).
- Chrome Root Store, Mozilla NSS, and Apple trust-store announcements.
- Go project `crypto/x509` advisory (CVE-2026-111943).
- Vendor analyses from Kudelski Security, Cloudflare, Help Net Security, Keyfactor, and Qualys.
- Reporting from The Hacker News.

Related references

- Certificate Trust Incident Response (the method this incident illustrates)
- Cryptography Reference
- CVE Reference
- Notable Incidents